
EMBARCADERO CONFERENCE
PATROCINADOR MASTER:
embarcadero
An Idera, Inc. Company

CACA AOS VAZAMENTOS: DOMINANDO MEMORY LEAKS NO DELPHI

WESLLEY CAPELARI

Um guia prático para identificar, prevenir e diagnosticar vazamentos de memória em aplicações Delphi (VCL/FMX e APIs REST) .



QUEM ESTÁ FALANDO

WESLLEY CAPELARI

Fiorilli Software

- → Graduando Engenharia de Software
- → Desenvolvendo software há mais de 7 anos

7+

ANOS DE CÓD



FIORILLI S/C SOFTWARE

DESDE 1974 · BÁLSAMO-SP

Especialista em software para o Setor Público — e é graças ao apoio dela que estou neste palco.

SCPI	Contabilidade
SIP	Folha e RH
SIA	Arrecadação
SIS	Saúde
SIE	Educação
SAS	Assistência Social

50+
anos de atuação — desde 1974

1.000+
municípios atendidos

2.755
clientes em todo o Brasil

1º
líder e a que mais cresce no segmento

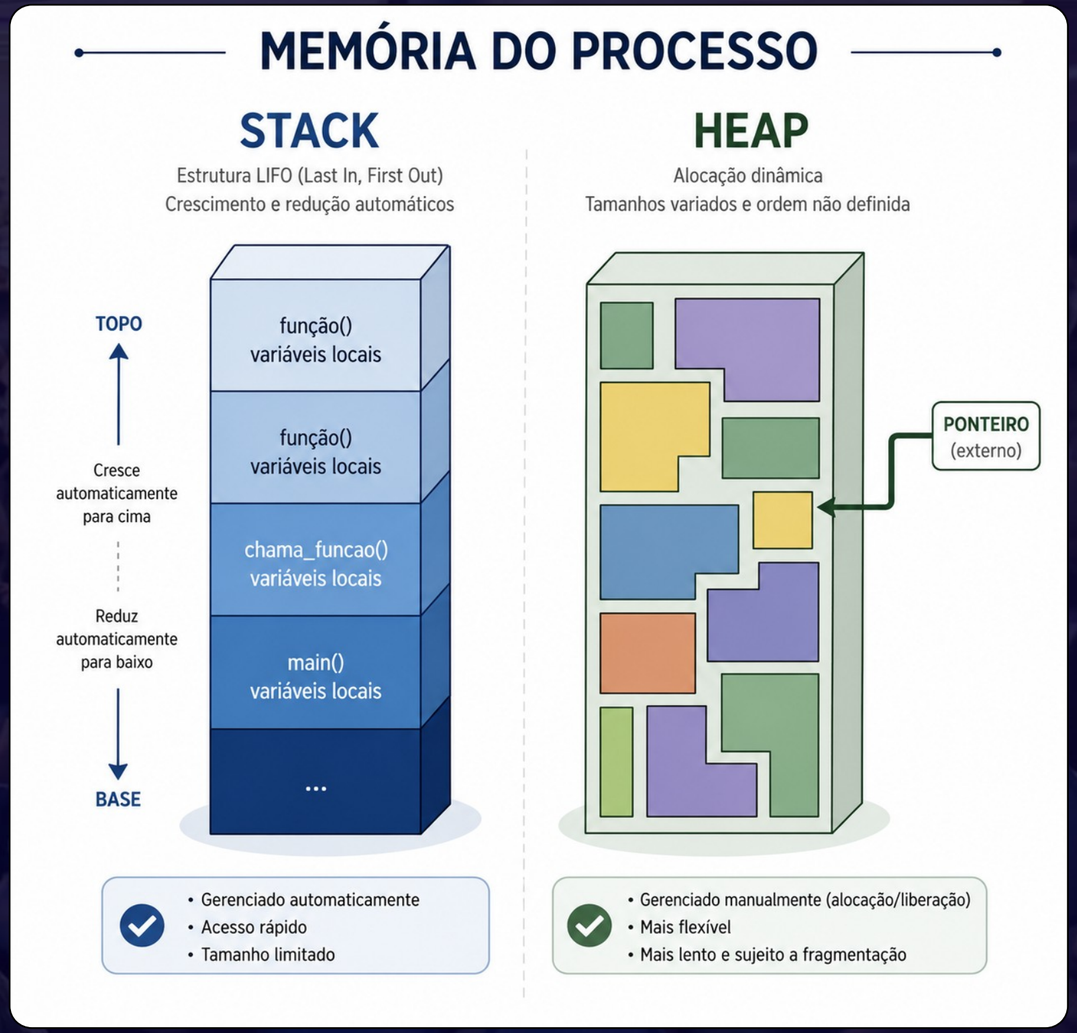
O IMPACTO SILENCIOSO DOS MEMORY LEAKS

- **Servidores caindo às 3h da manhã:** Out of Memory (OOM) em APIs/Serviços REST (Horse, DataSnap, RAD Server).
- **Degradação Progressiva:** O sistema fica lento com o passar das horas/dias de uso.
- **Desperdício de Recursos:** Aumento contínuo do consumo da RAM do servidor.
- **Difícil Reprodução em Dev:** Não ocorre com 1 usuário em teste de 5 minutos, mas explode em produção com 500 usuários.



🧠 ENTENDENDO A MEMÓRIA NO DELPHI: STACK VS HEAP

MEMÓRIA STACK (PILHA)	MEMÓRIA HEAP (MONTE)
Alocação rápida e automática	Alocação dinâmica via **GetMem** / **TObject.Create**
Gerenciada pelo escopo da função/método	Gerenciada **manualmente** pelo desenvolvedor
Variáveis locais (**Integer**, **Record**, etc.)	Instâncias de Objetos (**TObject**), Arrays Dinâmicos
Liberação automática ao sair do método	Exige chamadas explícitas de **Free** / **Dispose** / **FreeOf**



Memory Leak ocorre quando um bloco de memória alocado na **Heap** perde todas as suas referências sem ter sido devolvido ao Gerenciador de Memória.

⚠ ARMADILHA #1: ESQUECIMENTO DO TRY..FINALLY

O Código Perigoso ❌

```
procedure ProcessarDados;
var
  Lista: TStringList;
begin
  Lista := TStringList.Create;
  Lista.Add('Dados da Aplicação');
  // ... Se ocorrer um erro
  // ou esquecer o Free aqui -> LEAK!
end;
```

O Padrão Correto ✅

```
procedure ProcessarDados;
var
  Lista: TStringList;
begin
  Lista := TStringList.Create;
  try
    Lista.Add('Dados da Aplicação');
  finally
    // Garantido mesmo diante de exceções!
    Lista.Free;
  end;
end;
```

⚠ ARMADILHA #2: ALOCAÇÃO MÚLTIPLA INDEVIDA NO MESMO BLOCO

O Código Perigoso ❌

```
procedure ProcessarDados;
var
  ObjA, ObjB: TStringList;
begin
  ObjA := TStringList.Create;
  ObjB := TStringList.Create;
  // Se este Create falhar, ObjA VAZA!
  try
    // Operações com ObjA e ObjB
  finally
    ObjA.Free;
    ObjB.Free;
  end;
end;
```

O Padrão Correto ✅

```
procedure ProcessarDados;
var
  ObjA, ObjB: TStringList;
begin
  ObjA := TStringList.Create;
  try
    ObjB := TStringList.Create;
    try
      // Operações com ObjA e ObjB
    finally
      ObjB.Free;
    end;
  finally
    ObjA.Free;
  end;
end;
```

⚠ ARMADILHA #3: CONSTRUTORES QUE LANÇAM EXCEÇÃO

Se uma exceção ocorrer dentro do **constructor Create**, o Delphi **chama automaticamente o Destroy.**

Cuidado: Se você alocar ponteiros brutos (**GetMem**) no construtor **antes** da exceção, certifique-se de tratar a liberação no **Destroy** verificando **if Assigned(FBuffer).**

```
destructor TErroNoConstrutor.Destroy;
begin
  if Assigned(FBuffer) then
  begin
    FreeMem(FBuffer);
    FBuffer := nil;
  end;
  inherited Destroy;
end;
```



💻 LIVE DEMO 01: VAZAMENTOS BÁSICOS

- Demonstração de esquecimento de **Free**.
- Exceção sem **try..finally**.
- Alocação múltipla errada.
- Relatório do FastMM sendo exibido na tela no encerramento!

INTERFACES E REFERENCE COUNTING (ARC)

- Interfaces no Delphi herdam de **IInterface** (**_AddRef** e **_Release**).
- O Delphi gerencia a contagem de referências automaticamente!
- Quando **RefCount** chega a **0**, o objeto é destruído automaticamente.

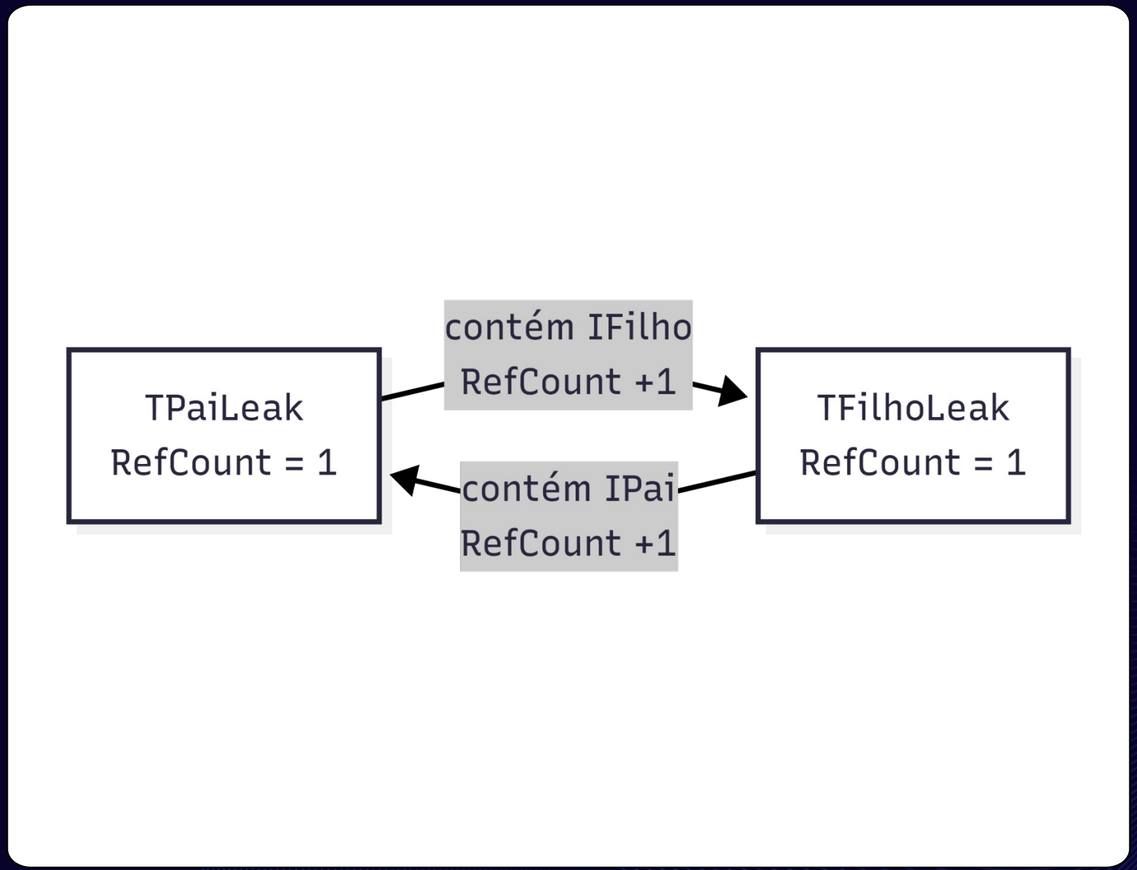
Atenção: O perigo mora nas **Referências Circulares**!

```
procedure ProcessarDados;
var
  MinhaIntf: IMyInterface;
begin
  // RefCount = 1
  MinhaIntf := TMyClass.Create;
  MinhaIntf.DoSomething;
end;
// Ao sair da scope
// MinhaIntf é nil
// RefCount = 0 -> Destruído!
```

⚠ ARMADILHA #4: REFERÊNCIA CIRCULAR ENTRE INTERFACES

O Resultado Desastroso: ambos chegam ao final do escopo com **RefCount = 1**. Nenhum dos dois chega a **0**.

Nenhum destruidor é chamado! Ambos ficam eternamente presos na memória Heap.



SOLUÇÕES PARA CICLOS DE INTERFACE

1. **Uso do Atributo [unsafe] / [weak]**
instrui o compilador a **não incrementar** a contagem de referência no membro filho

2. **Quebra Manual do Ciclo**
chamar explicitamente **Filho.SetPai(nil)** antes de descartar os objetos.

```
TFilhoFix = class(TInterfacedObject, IFilho)
Private
  // Não incrementa o RefCount do Pai!
  [unsafe] FPai: IPai;
public
  procedure SetPai(const APai: IPai);
end;
```



💻 LIVE DEMO 02: INTERFACES & REFERÊNCIA CIRCULAR

- Instanciação de **TPaiLeak** e **TFilhoLeak**.
- Prova de que os destruidores **NÃO** são chamados.
- Correção imediata utilizando **TPaiFix** e **TFilhoFix** (**[unsafe]**).

📦 COLEÇÕES & OWNERSHIP: GENERICS NO DELPHI

TList<T> armazena **ponteiros genéricos** — chamar **Lista.Free** ou **Lista.Clear** em uma **TList<TObject>** **NÃO** libera os elementos!

TList<T> ❌

```
procedure ProcessarDados;
var
  Lista: TList<TProduto>;
begin
  // LEAK: Os elementos TProduto
  // continuam no Heap!
  Lista := TList<TProduto>.Create;
  Lista.Add(TProduto.Create(1, 'Notebook'));
  Lista.Free;
end;
```

TObjectList<T> ✅

```
procedure ProcessarDados;
var
  Lista: TObjectList<TProduto>;
begin
  // OwnsObjects = True (Padrão)
  // libera a lista E os produtos!
  Lista := TObjectList<TProduto>.Create(True);
  Lista.Add(TProduto.Create(1, 'Notebook'));
  Lista.Free;
end;
```

🏛️ OWNERSHIP EM TCOMPONENT

O mecanismo de hierarquia do Delphi depende do parâmetro **AOwner**:

Create(nil) ❌

```
procedure ProcessarDados;
var
  Meubotao: TButton;
begin
  // LEAK: Owner é NIL! Ninguém vai destruir
  // o botão se você não der Free manual!
  Meubotao := TButton.Create(nil);
end;
```

Create(Self) ✅

```
procedure ProcessarDados;
var
  Meubotao: TButton;
begin
  // O Form (Self) assume a responsabilidade
  // de destruir o botão no seu Destroy!
  Meubotao := TButton.Create(Self);
end;
```

Dica de Ouro: Se passar **nil** no construtor de um **TComponent**, trate-o estritamente com **try..finally**!

🗂️ TDICTIONARY VS TOBJECTDICTIONARY

TDictionary<K, V> NÃO libera nem Chaves nem Valores por padrão!

TDictionary ❌

```
procedure ProcessarDados;
var
  Mapa: TDictionary<string, TProduto>;
begin
  Mapa := TDictionary<string, TProduto>.Create;
  Mapa.Add('PROD_1', TProduto.Create(100, 'Teclado'));
  Mapa.Free;
  // Libera o dicionario, MAS o TProduto vaza!
end;
```

TObjectDictionary ✅

```
procedure ProcessarDados;
var
  Mapa: TObjectDictionary<string, TProduto>;
begin
  Mapa := TObjectDictionary<string,
TProduto>.Create([doOwnsValues]);
  Mapa.Add('PROD_1', TProduto.Create(100, 'Teclado'));
  Mapa.Free;
  // Libera o dicionario E o TProduto!
end;
```

Modos disponíveis: **[doOwnsKeys]**, **[doOwnsValues]** e **[doOwnsKeys, doOwnsValues]**.



💻 LIVE DEMO 03: COLEÇÕES E COMPONENTES ÓRFÃOS

- Comparativo **TList<T>** vs **TObjectList<T>**.
- **TComponent.Create(nil)** vazando memória vs **TComponent.Create(Dono)**.
- Uso de **TObjectDictionary** com **[doOwnsValues]**.



MÉTODOS ANÔNIMOS, CLOSURES E MULTITHREADING

Armadilha com TThread.CreateAnonymousThread: quem é o dono do objeto capturado pela closure?

Solução: transfira o **Free** para **dentro** do bloco **try..finally** da própria thread.

```
procedure ProcessarDados;
var
  Dados: TWorkerData;
begin
  Dados := TWorkerData.Create('Payload');

  TThread.CreateAnonymousThread(
    procedure
    begin
      Sleep(1000);
      Writeln(Dados.Payload);
      // Se ninguem der Dados.Free -> LEAK!
      // Se der Dados.Free fora da
      // thread -> ACCESS VIOLATION!
    end).Start;
end;
```



TTHREAD CUSTOMIZADA: O RISCO DE FREEONTERMINATE

FreeOnTerminate é **False** por padrão em uma **TThread** derivada!

Se a thread terminar e ninguém chamar **Thread.Free** ou **Thread.WaitFor**, o objeto da thread vaza!

```
procedure ProcessarDados;
var
  Worker: TMyCustomThread;
begin
  // Para threads estilo "Fire and Forget":
  Worker := TMyCustomThread.Create(True);
  Worker.FreeOnTerminate := True;
  // A thread se auto-destruirá ao concluir!
  Worker.Start;
end;
```



💻 LIVE DEMO 04: THREADS E ANONYMOUS METHODS

- Thread customizada sem **FreeOnTerminate** vazando a instância.
- Captura assíncrona de objeto com liberação interna segura.

🔬 FERRAMENTA #1: FASTMM4 / FASTMM5 NATIVO

Habilitando o diagnóstico nativo — basta adicionar no seu arquivo **.dpr**:

```
program MeuProjeto;

uses
  Vcl.Forms,
  uMain in 'uMain.pas';

begin
  // Habilita o popup/relatório de
  // leaks ao fechar a aplicação
  ReportMemoryLeaksOnShutdown := True;

  Application.Initialize;
  Application.MainFormOnTaskbar := True;
  Application.CreateForm(TFormMain, FormMain);
  Application.Run;
end.
```

♂ FERRAMENTA #2: FASTMM FULL DEBUG MODE + STACK TRACE

Para descobrir a **linha exata do código** onde o leak foi alocado:

1. Ative a flag **DetailedMemoryLeakReport** no **FastMM4Options.inc** ou use o **FastMM5**.
2. No Delphi IDE: vá em **Project Options > Building > Delphi Compiler > Linking**.
3. Defina **Map File** como **Detailed** (**.map**).
4. Coloque a DLL **FastMM_FullDebugMode.dll** na pasta do executável.

Resultado: o relatório do FastMM gera um arquivo **.txt** com a **árvore de chamadas (Call Stack)** completa, com arquivo **.pas** e número da linha!



💻 LIVE DEMO 05: FASTMM E O PADRÃO RAII/GUARD

- Padrão RAII/Guard liberando um objeto automaticamente ao sair de escopo.
- Vazamento proposital com **GetMem**/**FreeMem** esquecido.
- Relatório do FastMM apontando exatamente o bloco não liberado.

🛠️ FERRAMENTAS AVANÇADAS DE MERCADO

FERRAMENTA	TIPO	PONTO FORTE
FastMM5	Gerenciador de Memória	Altíssimo desempenho multithread e relatórios ricos em runtime.
Deleaker	Profiler / Extension RAD Studio	Análise em tempo real dentro do IDE para Leaks, GDI, Handles e Heaps.
AQTime Pro	Profiler Completo	Deep profiling de memória, CPU e cobertura de código.
EurekaLog / MadExcept	Crash Reporting	Captura leaks e exceções não tratadas em ambientes de teste/clientes.



PADRÃO RAII NO DELPHI

Usando a contagem de referência de Interfaces locais para criar **Guards** que liberam objetos automaticamente ao sair de escopo (Smart Pointers no Delphi):

```
procedure ProcessarDados;
var
  Guard: ILocalGuard<TStringList>;
  Lista: TStringList;
begin
  Guard := TLocalGuard<TStringList>.Create(TStringList.Create);
  Lista := Guard.Target;

  Lista.Add('Item Protegido');
  // Não precisa de try..finally!
  // Ao sair do método, a Interface Guard
  // é zerada e o destruidor do Guard
  // libera a Lista automaticamente.
end;
```

📋 CHECKLIST ANTI-LEAK

1️⃣ Criou um **TObject**? Coloque **imediatamente** o **try..finally** antes de escrever qualquer outra linha.
2️⃣ Alocou múltiplos objetos? Use blocos **try** aninhados ou **TObjectList**.
3️⃣ Usando **TList<T>**? Substitua por **TObjectList<T>** com **OwnsObjects = True**.
4️⃣ Usando **TDictionary**? Substitua por **TObjectDictionary** com **[doOwnsValues]**.
5️⃣ **TComponent** criado em runtime? Verifique se passou o **Owner** correto ou trate a liberação manual.
6️⃣ Interfaces cruzadas? Utilize **[unsafe]** ou **[weak]** para quebrar referências circulares.
7️⃣ Threads assíncronas? Defina **FreeOnTerminate := True** ou garanta o **Free** dentro da thread.
8️⃣ Sempre compile em Debug com **Map File Detailed** ativado nos ambientes de teste.
9️⃣ Deixe **ReportMemoryLeaksOnShutdown := True** ativo em Builds de Debug/Homologação.
🔟 Monitore o **Working Set** de RAM dos seus servidores REST/Horse em produção.



? PERGUNTAS & RESPOSTAS (Q&A)

- Padrão RAII/Guard liberando um objeto automaticamente ao sair de escopo.
- Vazamento proposital com **GetMem**/**FreeMem** esquecido.
- Relatório do FastMM apontando exatamente o bloco não liberado.

EMBARCADERO CONFERENCE 26

PATROCINADOR MASTER: embarcadero® An Idera, Inc. Company

QUER ME VER NA
#ECON27?
ACESSE O QRCODE E
AVALIE MINHA PALESTRA!

WESLLEY CAPELARI

@weslley.capelari

linkedin.com/in/weslley-capelari

weslley.capelari@gmail.com

(18) 9 9602-4649